StackGraph

Exploratory Network Science case study | First professor meeting

Research question

To what extent can dependency-graph information improve the identification of stacked pull-request workflows compared with event-based heuristics, under constrained GitHub API access?

Evidence already obtained

One selected case: eight public PRs from cli/cli, one author; four verified branch-targeting edges form a five-PR chain, with three isolates and no cycles. The 60-minute heuristic yields groups of sizes 2 and 6, mixing one control with the chain. Native membership documents the same five-PR stack. At 15/60/180 minutes, the heuristic gives 3/2/1 groups. There are 32 passing tests and reproducible offline execution. These are descriptive observations, not validated detection accuracy.

Three distinct contributions to test

A. Information value: do dependency fields improve on event timing? B. Algorithm value: do connectivity, reachability, ordering, or cycle checks improve on rules given identical fields? C. Budget value: what reconstruction and coverage remain at fixed request budgets? The case illustrates added structural context; none of these benefits is independently validated.

Proposed comparison; not yet executed

E: event-only heuristic. M: enriched metadata rules with direct repository/ref matching, without global graph algorithms. G: identical enriched metadata and matching rule, plus graph aggregation/analysis. Compare M with E for information value, and G with M for incremental algorithm value; repeat at fixed budgets with the same acquisition schedule for M/G. Equal performance is a valid outcome. Native membership remains descriptive context, not independent ground truth. E is not AgentScan's production classifier.

Proposed metrics and independent labels

Stack-membership precision/recall on unordered same-stack PR pairs; dependency-edge precision/recall on directed adjacent prerequisites; API calls per exactly reconstructed stack; and decision/field/discovery coverage under missing information. Predefine denominators and abstentions. Obtain independent maintainer or code/history adjudication, blind to outputs/native labels. Keep uncertain labels unknown. None of these performance metrics has been measured here.

Practical constraints and motivation

GitHub activity timelines expose up to 300 events within 30 days [1]. Needed stack information may be unavailable in activity payloads; the schema references a nested PR object but does not explicitly specify stack membership [2]. Additional requests consume budget [3]; the original case saved eight PR-detail and eight stack-lookup responses, not a minimum-cost benchmark. Discord feedback about false negatives is presenter-reported private communication without supplied cases/transcript; it motivates evaluation and supplies no labels.

Limits and next decision

Selection used native membership and branch information; one repository/author and incomplete event sampling limit inference. Branches/memberships are mutable snapshots; verified arrows establish targeting, not semantic necessity. No direct stack-field leakage was found, but native/graph conceptual overlap and selection bias remain. Accuracy, incremental algorithm benefit, and budget efficiency are unproven. Structure establishes neither legitimacy nor human authorship. Agree on the target and a modest independently labeled follow-up before extending implementation.

[1] GitHub REST events [2] GitHub event types [3] GitHub REST rate limits
Exact links/evidence: claim_support.md. Metric definitions/proposal: evaluation_plan.md. Prior graph retained.